좁은 트랙 자율주행 레이싱을 위한 위치추정 비 의존 원호 회랑 장애물 감지

Pose-Independent Arc-Corridor Obstacle Detection for Autonomous Racing on Narrow Tracks

이준희 · 김형준†

Joonhee Lee[1], Hyeongjun Kim†

Abstract: Obstacle detection for 1/10-scale autonomous racing usually depends on the estimated pose, because scans are compared with the map or with the planned path. On tracks 1.2 to 1.6 m wide, the position covariance reported by AMCL on our car corresponded to a standard deviation of 0.3 to 0.45 m, close to the lane half-width of 0.6 m, and pose-dependent detectors kept mistaking walls for obstacles. We describe an arc-corridor detector that needs no pose. It predicts the driving arc from the last steering command and counts only the LiDAR points inside a corridor of 0.22 m half-width along that arc, over a look-ahead distance that grows with speed. A two-stage supervisor first limits the speed and hands control to reactive avoidance only when the detection persists. On the track of the 29th RoboRacer competition, the car recorded a best practice lap of 14.7 s with a top speed of 3.65 m/s. In closed-loop simulation, raising the lateral acceleration limit from 2.2 to 6 m/s^2 shortened the lap from 15.3 s to 11.3 s. A heartbeat node missing from the startup procedure kept the car from starting in qualification, which showed that a reliable startup matters as much as the algorithm.

Keywords: Autonomous racing, RoboRacer, Obstacle detection, Pure pursuit, 2D LiDAR

1. 서 론

RoboRacer(구 F1TENTH)는 1/10 스케일 차량으로 겨루는 자율주행 레이싱 대회다[1]. 2026년 8월 부산 벡스코에서 열린 제29회 대회(IFAC World Congress 2026)는 약 8 m × 22 m 구역에 통로 폭 1.1-1.6 m인 트랙을 깔았다. 주행 중에는 사람이 개입할 수 없고, 본선은 임의로 놓인 장애물과 상대 차량이 있는 일대일 경주였다. 이런 좁은 트랙에서는 장애물을 놓치는 것만큼 벽을 장애물로 잘못 보는 오발동이 문제가 된다. 오발동이 나면 차가 주행선을 버리고 회피 모드로 넘어가 랩타임과 안정성을 함께 잃는다.

맵에 없는 물체를 찾는 방식과 계획 경로 주변만 살피는 방식은 모두 차량의 추정 위치를 써야 한다. 우리 차량에서 AMCL[2]이 보고한 위치 공분산은 약 0.2 m^{2}로 표준편차 0.3-0.45 m에 해당했고, 이는 차선 반폭 0.6 m에 가까운 값이다.

차량 좌표계에서 원호를 따라 충돌을 검사하는 방법은 오래전부터 쓰였다. Dynamic Window Approach[3]는 속도와 각속도 조합이 만드는 원호마다 장애물까지의 거리를 검사하고, Tentacles[4]는 곡률이 다른 원호 다발의 주변만 살핀다. Regulated Pure Pursuit[5]는 현재 명령을 앞으로 투영한 원호를 코스트맵에서 검사한다. 이 논문의 방법은 이미 정해진 조향 명령 하나의 원호만 LiDAR 스캔 점에서 직접 검사한다. 계산이 가볍고 추정 위치와 맵이 필요 없다.

2. 시스템 구성

차량은 Traxxas 1/10 4WD 섀시에 NVIDIA Jetson Orin

※ This work was supported by the RISE Project Group (Future Mobility and Robotics) of Gyeongsang National University

1. Undergraduate Student, Future Automotive Engineering, Gyeongsang National University, Jinju, Korea (junehee0207@gnu.ac.kr)

† Professor, Corresponding author: Future Automotive Engineering, Gyeongsang National University, Jinju, Korea (hj.kim@gnu.ac.kr)

NX 16GB, VESC 6 모터 제어기, Hokuyo UST-10LX 2D LiDAR(270°, 1,081빔, 40 Hz)를 실었고 ROS 2로 구동한다. 지도는 Cartographer[6]로 만들고 위치는 AMCL로 추정한다.

주행은 세 노드가 나눠 맡는다. 평소에는 Pure Pursuit[7]가 웨이포인트를 따라가며, 곡률로 정한 속도 프로파일에 따라 코너 앞에서 미리 감속한다. 장애물이 있으면 Follow-the-Gap[8]이나 Disparity Extender[9]가 LiDAR만으로 회피 조향을 낸다. 감독 노드는 두 명령 가운데 하나를 골라 우선순위 mux를 거쳐 VESC로 보낸다.

Pure Pursuit는 처음에 AMCL 위치가 갱신될 때만 조향을 계산했다. 주행 기록을 보니 이 갱신이 최대 2.2 s 끊긴 구간과 코너 충돌이 겹쳤다. 조향 계산을 TF(map→base_link) 기반 25 Hz 고정 주기로 바꿔 AMCL 보정 사이를 40 Hz 오도메트리가 메우게 했다.

3. 원호 회랑 감지

3.1 위치에 의존하는 방식의 한계

처음에는 전방 2.0 m 구간의 스캔 점을 맵 좌표로 옮겨 점유 격자와 비교했고, 다음에는 앞으로 달릴 경로 2.5 m 주변의 튜브 안에 들어온 점을 장애물로 봤다. 두 방식 모두 시뮬레이션에서는 오발동이 없었지만 실제 트랙에서는 벽을 장애물로 판정했다. 위치 오차만큼 맵과 튜브가 스캔에서 어긋나기 때문이다. 장애물이 시야를 가리면 AMCL 공분산이 0.1에서 12까지 커지고 회복에 7-13 s가 걸려, 감지가 필요한 순간에 위치가 가장 부정확했다.

3.2 방법

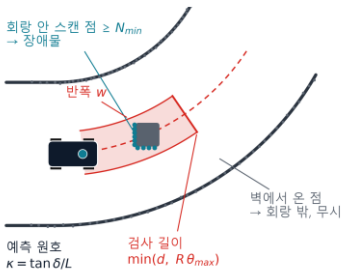
제안 방법은 차량 좌표계의 스캔 점과 마지막 조향 명령 δ만 쓴다. 자전거 모델에서 곡률은 $\kappa = \tan\delta / L$(L은 축간거리)이고, 차량이 반지름 $R = 1/|\kappa|$인 원호를 따라 간다고 예측한다. 스캔 점 p_i와 원호 중심 사이의 거리를 r_i, 원호를 따라 잰 거리를 s_i라 할 때 식 (1)을 만족하는 점을 회랑 안의 점으로 센다[Fig. 1].

$$|r_i - R| \le w, \quad 0 < s_i \le \min(d, R\theta_{\max}) \tag{1}$$

w는 회랑 반폭(0.22 m), $\theta_{\max}$는 원호 각도 상한이다. 감지 거리 d는 식 (2)처럼 속도 v에 따라 늘린다.

$$d = 0.45 + 0.55\,v \quad [\mathrm{m}] \tag{2}$$

회랑 안의 점이 $N_{\min}$개 이상이면 장애물 후보로 본다. 회랑이 주행선과 함께 휘므로 코너 벽은 회랑 밖에 남는다. 코너를 지나 조향이 풀리면 예측 원호가 실제 경로에서 벗어나 벽을 훑으므로 $\theta_{\max}$로 검사 구간을 자른다. 식 (2)의 계수와 $\theta_{\max}$, $N_{\min}$은 주행선 전 구간에서 벽만 있을 때 오발동이 0건인 가장 긴 감지 거리를 찾아 정했다.



[Fig. 1] Arc-corridor detection. Only the scan points inside the corridor of half-width w along the arc predicted from the last steering command are counted.

3.3 2단계 대응

후보가 잡혀도 감독 노드는 곧바로 회피로 넘어가지 않는다. 먼저 Pure Pursuit를 유지한 채 속도만 1.2 m/s로 0.5 s 동안 제한하고(CAUTION), 후보가 3-5 스캔(75-125 ms) 이어질 때만 회피 노드로 넘긴다(AVOID). 오발동의 대가는 주행선 이탈에서 0.5 s 감속으로 줄고, 실제 장애물 앞에서는 확인하는 동안 감속이 먼저 시작된다.

3.4 트랙별 보정

통로 폭이 1.2 m로 좁은 트랙에서는 긴 원호가 코너 벽에 닿아 $\theta_{\max}$와 w를 줄이고 $N_{\min}$을 늘려야 했다[Table 1]. 트랙별로 모은 스캔으로 검증한 결과 두 트랙 모두 주행선 위에서 벽 오발동이 0건이었다. 연습 트랙 1에서는 직선의 장애물을 1.55 m 앞에서, 연습 트랙 2에서는 장애물 6개를 모두 0.72-0.80 m 앞에서 감지했다. 웨이포인트를 통로 중앙으로 다시 맞추자 횡편차 ±0.25 m일 때의 벽 오발동이 19%에서 3%로 줄었다.

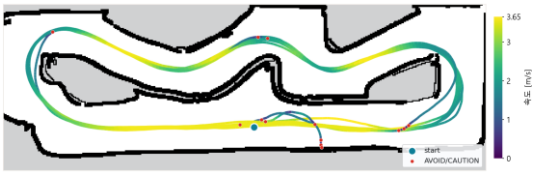
[Table 1] Detection parameters for each practice track

Track	Width	w	$\theta_{\max}$	$N_{\min}$
Practice 1	1.5 m	0.22 m	0.6 rad	15
Practice 2	1.2 m	0.16 m	0.3 rad	25

4. 실험 결과

4.1 대회 트랙 실주행

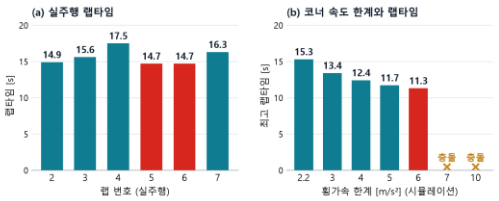
2026년 8월 25일 대회 트랙에서 속도 상한 4.0 m/s로 달린 연습 주행을 rosbag으로 기록해 분석했다[Fig. 2]. 출발 랩을 뺀 6랩의 랩타임은 14.7-17.5 s(평균 15.6 s)였고 최고 속도는 3.65 m/s였다[Fig. 3(a)]. 명령 속도에 대한 실제 속도의 비는 1.00, 벽까지의 평균 여유는 0.74 m였다. 가장 느린 17.5 s 랩에는 회피 전환이 끼어 있었고, CAUTION과 AVOID는 코너 출구와 시케인에 몰렸다.



[Fig. 2] Seven practice laps on the competition track. Color shows speed; red dots mark where CAUTION or AVOID was active.

4.2 속도 한계 분석

실차 노드를 그대로 돌리는 페루프 시뮬레이터를 만들고, 실주행 기록에서 얻은 속도 지연 0.28 s와 가감속 한계를 차량 모델에 넣었다. Pure Pursuit의 목표 속도는 속도 상한 v_{max}, 횡가속 한계 $a_{y,max}$가 정하는 코너 속도, 가감속 프로파일 가운데 가장 작은 값이다. v_{max}를 3.5에서 4.0 m/s로 올리면 최고 랩타임이 15.2 s에서 14.4 s로 줄었지만 차이는 직선에서만 났다. $a_{y,max}$를 2.2에서 6.0 m/s^{2}로 올리면 15.3 s에서 11.3 s로 26% 줄었고, 7.0 이상에서는 코너에서 벽에 부딪혔다[Fig. 3(b)]. 두 실험은 감속 한계 설정이 달라 기준 랩타임이 다르다. 이 시뮬레이터에는 타이어 슬립이 없어 실차 한계는 이보다 낮다.



[Fig. 3] (a) Lap times of the six practice laps. (b) Best simulated lap time for each lateral acceleration limit (wall hit from 7 m/s^2).

5. 결 론

추정 위치를 쓰지 않는 원호 회랑 감지와 2단계 대응으로 폭 1.2-1.5 m인 연습 트랙에서 벽 오발동 없이 주행선 위의 장애물을 감지했고, 대회 트랙에서 최고 14.7 s의 랩타임을 기록했다.

한계도 확인했다. 코너 출구에서는 조향이 아직 남아 있어 회랑이 직선 위의 장애물을 비껴간다. 정면 0.6 m를 항상 살피는 범퍼 존을 더했지만 감지 거리가 0.65-0.70 m로 짧다. 예선에서는 비상정지 잠금을 푸는 하트비트 노드가 기동 절차에서 빠져 mux가 모든 주행 명령을 막았고, 원인을 찾는 동안 예선 시간이 지나 기록을 남기지 못했다. 앞으로 기동 절차를 자동 점검으로 바꾸고, 조향이 풀린 직선 예측을 함께 검사해 코너 출구의 사각을 줄이며, 타이어 모델을 넣은 시뮬레이션과 실차 시험으로 횡가속 한계를 확인할 계획이다.

References

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning," *Proceedings of the NeurIPS 2019 Competition and Demonstration Track*, PMLR Vol. 123, pp. 77-89, 2020.

[2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.

[3] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics & Automation Magazine*, Vol. 4, No. 1, pp. 23-33, 1997.

[4] F. von Hundelshausen, M. Himmelsbach, F. Hecker, A. Mueller, and H.-J. Wuensche, "Driving with Tentacles: Integral Structures for Sensing and Motion," *Journal of Field Robotics*, Vol. 25, No. 9, pp. 640-673, 2008.

[5] S. Macenski, S. Singh, F. Martin, and J. Gines, "Regulated Pure Pursuit for Robot Path Tracking," *Autonomous Robots*, Vol. 47, pp. 685-694, 2023.

[6] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time Loop Closure in 2D LIDAR SLAM," *IEEE International Conference on Robotics and Automation*, Stockholm, Sweden, pp. 1271-1278, 2016.

[7] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," Carnegie Mellon University, Pittsburgh, USA, Rep. CMU-RI-TR-92-01, 1992.

[8] V. Sezer and M. Gokasan, "A Novel Obstacle Avoidance Algorithm: Follow the Gap Method," *Robotics and Autonomous Systems*, Vol. 60, No. 9, pp. 1123-1134, 2012.

[9] N. Otterness, The Disparity Extender Algorithm, and F1/Tenth, [Online], https://www.nathanotterness.com/2019/04/the-disparity-extender-algorithm-and.html, Accessed: Nov. 11, 2026.